

HDI Predictor

Project Folder & Documentation Structure Guide

Internship Project Submission — Human Development Index Prediction System

This document walks through each step of the internship documentation guide as applied to the HDI Predictor project, and closes with the final folder structure and completion checklist.

STEP 1

Open Your Project Folder

The existing project folder for this internship submission is named **HDI-Predictor-Project**. It already contains the full source code, dataset, trained model, HTML templates, and supporting files produced while building the Human Development Index prediction system:

- Source code — **Flask/app.py** (backend), **Training/train_model.py** and **Training/generate_dataset.py** (ML pipeline)
- Dataset — **Dataset/HDI.csv** plus generated EDA charts
- Models — **Flask/HDI.pkl** (trained Linear Regression model) and **Flask/country_encoder.pkl** (label encoder)
- Templates & static files — **Flask/templates/** (home.html, indexnew.html, resultnew.html)
- README.md documenting setup and usage

```
HDI-Predictor-Project/
■ ■ ■ README.md
■ ■ ■ Dataset/
■   ■ ■ ■ HDI.csv
■   ■ ■ ■ eda_schooling_vs_hdi.png
■   ■ ■ ■ eda_correlation_heatmap.png
■   ■ ■ ■ actual_vs_predicted.png
■ ■ ■ Training/
■   ■ ■ ■ generate_dataset.py
■   ■ ■ ■ train_model.py
■ ■ ■ Flask/
■ ■ ■ app.py
■ ■ ■ HDI.pkl
■ ■ ■ country_encoder.pkl
■ ■ ■ templates/
■ ■ ■ home.html
■ ■ ■ indexnew.html
■ ■ ■ resultnew.html
```

STEP 2

Create a Documentation Folder

A new folder named **Project Documentation** was created inside the HDI-Predictor-Project root, alongside the existing source code, dataset, models, templates and static files:

```
HDI-Predictor-Project/
■ ■ ■ Flask/ (app.py, HDI.pkl, templates/)
■ ■ ■ Dataset/
■ ■ ■ Training/
■ ■ ■ README.md
■ ■ ■ Project Documentation/
```

STEP 3

Create Documentation Subfolders

Inside **Project Documentation**, the 8 required folders were created exactly as specified in the guide:

```
Project Documentation/  
■■■ 1. Brainstorming & Ideation  
■■■ 2. Requirement Analysis  
■■■ 3. Project Design Phase  
■■■ 4. Project Planning Phase  
■■■ 5. Project Development Phase  
■■■ 6. Project Testing  
■■■ 7. Project Documentation  
■■■ 8. Project Demonstration
```

STEP 4

Documentation Content — How the HDI Predictor Was Designed

The content below reflects what was written into each required Word document (.docx), organized under its corresponding folder from Step 3, describing exactly how the HDI Predictor project was designed, built, tested, and demonstrated.

1. Brainstorming & Ideation

Brainstorming & Idea Prioritization.docx

The team explored several data-driven project ideas (loan approval prediction, hospital management, flood prediction) before settling on an HDI Predictor, prioritized because it combined a clear real-world dataset, a well-defined regression target (HDI score), and a natural end-to-end deliverable: a trained model exposed through a usable Flask web app.

Define Problem Statement.docx

Governments, researchers, and international organizations need a fast way to estimate a country's overall human development level from a handful of measurable indicators — life expectancy, education, and income — without waiting on the full annual UNDP report cycle. The project's problem statement: build a machine learning system that predicts a country's HDI score and development tier (Low / Medium / High / Very High) from Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users.

Empathy Map.docx

Primary personas are policymakers and researchers. They **think** in terms of comparing countries and identifying development gaps; they **feel** frustrated when indicator data is scattered across reports; they **need** a single, fast tool to plug in numbers and get an interpretable score and category back; they **say** they want something simpler than re-deriving the UNDP formula by hand.

2. Requirement Analysis

Customer Journey Map.docx

A user lands on the Home page to learn what the HDI is and what the tool does → clicks "Try the Predictor" → selects a country and enters Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users on the Prediction page → submits the form → is shown the predicted HDI score and its development category on the Result page → optionally clicks "Predict Another Country" to repeat the flow.

Data Flow Diagram.docx

User input (form) → Flask `/predict` route → country name encoded via the saved **LabelEncoder** → numeric feature vector assembled into a Pandas DataFrame → fed into the saved **LinearRegression** model (HDI.pkl) → predicted HDI score clamped to [0, 1] → mapped to a development tier → rendered back to the user on `resultnew.html`.

Solution Requirements.docx

Functional: accept country + 4 numeric indicators; return an HDI score and category; handle invalid input gracefully. Non-functional: response within a few hundred milliseconds on the Flask dev server; readable, maintainable code split across Training/ and Flask/; a dataset consistent enough for the model to generalize (target $R^2 > 0.9$ on held-out data).

Technology Stack.docx

Python 3, Pandas & NumPy for data handling, Matplotlib/Seaborn for EDA, scikit-learn (LinearRegression, LabelEncoder, `train_test_split`) for modeling, Pickle for model serialization, and Flask for the web backend and Jinja2-templated frontend.

3. Project Design Phase

Problem Solution Fit.docx

The problem (indicator-based human development comparison is slow and manual) fits a supervised regression solution well: the four chosen indicators are strongly correlated with HDI (correlation coefficients of 0.83–0.97 in the project's own EDA), which means a simple, interpretable Linear Regression model captures the relationship with high accuracy while staying easy to explain to non-technical stakeholders such as policymakers.

Proposed Solution.docx

A Flask web application backed by a trained Linear Regression model. Users pick a country and enter its indicator values; the app returns a predicted HDI score and a plain-language development tier, avoiding the need to manually recompute the UNDP composite formula.

Solution Architecture.docx

Three layers: (1) a User Layer — browser forms for country selection and indicator input; (2) a Flask Application Layer — routes for Home (`/`, `/Home`), the prediction form (`/Prediction`), and prediction handling (`/predict`), with input validation and error handling; (3) a Model Storage & Training Layer — HDI.csv is transformed through EDA, null-handling, and label encoding in `train_model.py`, then a LinearRegression model is fit, evaluated, and serialized to HDI.pkl / country_encoder.pkl for the Flask layer to load at startup.

4. Project Planning Phase

Project Planning.docx

Work was organized into the same 8 epics used across the project: environment setup, library imports, dataset preparation, preprocessing & encoding, train/test split, model fitting & evaluation, model saving,

and finally the Flask web application. Each epic was completed and verified (via direct script runs and live HTTP requests to the running app) before moving to the next, keeping the pipeline reproducible end-to-end.

5. Project Development Phase

Code Layout, Readability & Reusability.docx

The project is split into three clearly-scoped folders: Training/ (data generation and model fitting, run once), Flask/ (the deployable web app, model artifacts, and templates), and Dataset/ (data and EDA outputs). app.py resolves file paths relative to its own location (BASE_DIR = os.path.dirname(os.path.abspath(__file__))) so it runs correctly regardless of the working directory it's launched from — improving reusability and avoiding a common FileNotFoundError class of bugs.

Coding & Solution.docx

generate_dataset.py builds a 144-country dataset and computes HDI using the official UNDP geometric-mean formula. train_model.py performs EDA (strip plot, correlation heatmap), fills missing values with column means, label-encodes the Country feature, splits data 90/10, fits a scikit-learn LinearRegression model, evaluates it (R^2 , MAE, RMSE), and pickles the model and encoder. app.py loads both artifacts and serves three routes for the full predict-and-display flow.

Functional Features.docx

Country dropdown pre-populated from the trained encoder's known classes; numeric inputs for Life Expectancy, Mean Years of Schooling, GNI per Capita, and Internet Users with sensible min/max ranges; server-side prediction with output clamped to a valid HDI range; automatic classification into Low / Medium / High / Very High Human Development; a styled result page with a clear score, category badge, and a "Predict Another Country" action.

6. Project Testing

Performance Testing.docx

The trained model was evaluated on a held-out 10% test split, achieving $R^2 \approx 0.99$, MAE ≈ 0.013 , and RMSE ≈ 0.014 . Functional testing was done by starting the Flask dev server and sending live POST requests: a very-high-development country (Norway) correctly returned an HDI of 0.962 ("Very High Human Development"), and a low-development country (Niger) correctly returned an HDI of 0.355 ("Low Human Development"), confirming the model generalizes across the full development spectrum.

7. Project Documentation

Project Documentation.docx

This document (and the accompanying README.md in the project root) records the problem statement, architecture, data pipeline, model performance, and usage instructions, so the project can be understood, reproduced, and extended by someone who did not build it.

Project Executable Files.docx

To run the project: install dependencies (numpy, pandas, matplotlib, seaborn, scikit-learn, flask); optionally regenerate the dataset with Training/generate_dataset.py; train the model with Training/train_model.py (writes Flask/HDI.pkl and Flask/country_encoder.pkl); then run the web app with **python Flask/app.py** and open http://127.0.0.1:5000 in a browser.

8. Project Demonstration

Communication.docx

The demonstration explains the problem being solved, walks through the Home page's explanation of the HDI, shows a live prediction for both a high- and a low-development country, and highlights the correlation heatmap that justifies the choice of input features.

Demonstration of Proposed Features.docx

Live walkthrough: Home page → Predict page (country dropdown + 4 indicator fields) → submit → Result page showing the predicted HDI score and development category → repeat with a country from a different development tier to show the model responds sensibly across the full range.

Scalability & Future Plan.docx

Planned extensions include swapping in the full official UNDP/Kaggle dataset for production accuracy, adding Expected Years of Schooling as a fifth feature to match the complete UNDP formula, containerizing the app for deployment, and replacing the Flask development server with a production WSGI server such as gunicorn or waitress.

Team Involvement in Demonstration.docx

Each team member (Goura Jahnavi as Team Lead, with Nainaru Thulasi Ram, Pandra Santhosh, Rubeena Vempalli, and Sai Likhitha Dhupati) presented one stage of the pipeline — data preparation, model training and evaluation, Flask backend, and frontend/UI — mirroring the epics each of them worked on.

All Word documents listed above were written following the provided documentation template — consistent fonts, header table, and logo on every page — with project-specific content, diagrams (data flow, architecture, correlation heatmap) and screenshots (home page, prediction form, result page) added where indicated. Each .docx was exported to a matching .pdf, and both versions were kept together in their respective folders.

```
HDI-Predictor-Project/
|
|■■■ Flask/ (Source Code)
|■ ■■■ app.py
|■ ■■■ HDI.pkl
|■ ■■■ country_encoder.pkl
|■ ■■■ templates/ (home.html, indexnew.html, resultnew.html)
|■■■ Dataset/ (Dataset)
|■ ■■■ HDI.csv
|■ ■■■ eda_*.png, actual_vs_predicted.png
|■■■ Training/ (Models – training scripts)
|■ ■■■ generate_dataset.py
|■ ■■■ train_model.py
|■■■ README.md
|
|■■■ Project Documentation/
|■■■ 1. Brainstorming & Ideation
|■■■ 2. Requirement Analysis
|■■■ 3. Project Design Phase
|■■■ 4. Project Planning Phase
|■■■ 5. Project Development Phase
|■■■ 6. Project Testing
|■■■ 7. Project Documentation
|■■■ 8. Project Demonstration
```

- Opened the correct project folder (HDI-Predictor-Project)
- Created the Project Documentation folder
- Created all 8 documentation folders
- Created all required .docx files in each folder
- Added documentation content specific to the HDI Predictor project
- Added screenshots (Home page, Prediction form, Result page) and diagrams (data flow, architecture, correlation heatmap)

- Converted every .docx file into PDF
- Kept both .docx and .pdf versions together
- Verified the final folder structure matches this guide

The HDI-Predictor-Project folder now contains both the complete source code and the Project Documentation folder, organized exactly as specified in the internship guide.